2.2 Designing Algorithms

Sue just spent a half hour writing the code for a certain function before she realized that she got the design all wrong. Not only was her code broken, but her entire approach to the problem was all wrong. Sue is frustrated: writing code is hard! If only there was a way to design a function without having to go through the work of getting it to compile…

Objectives

By the end of this class, you will be able to:

- Recite the pseudocode keywords.
- Understand the degree of detail required for a pseudocode design.
- Generate the pseudocode corresponding to a C++ function.

Prerequisites

Before reading this section, please make sure you are able to:

- Create a map of a program using structure charts (Chapter 2.0).

Reading

All of the reading in this section will consist of the pseudocode videos:

- Design First: This video discusses why it is important to design a problem before the implementation is begun.
- When to Design: This video will discuss different times in the software development process when design activities are most effective.
- Different Design Approaches: This video will introduce four techniques to drafting a design: the paragraph method, flowchart, structure diagram, and pseudocode.
- Using Pseudocode: How pseudocode can be used as a tool to more effectively write software.
- Rules of Pseudocode: The conventions and rules of pseudocode.
- Pseudocode Keywords: The seven classes of keywords that are used in pseudocode.

Another design tool

Pseudocode, along with the Structure Chart, is one of our most powerful design tools. While the Structure Charts is concerned with what function we have in our program and how the functions interact with each other, Pseudocode is concerned with what goes on inside individual functions.

There are several reasons why we need to draft a solution before we start writing code. First, we need to be able to think big before getting bogged down in the details. Programming languages resist this process; they need you to always work at the most detailed level. This can be very frustrating; you are trying to solve a large problem but the language keeps forcing you to specify data types and work through syntax errors.

Using pseudocode

Pseudocode is a tool helping the programmer bridge the high-level English description of a problem and the C++ solution. To illustrate how this works, we will begin with a natural language definition of a problem. In this case, how to compute your tax liability with a simple 2 tier tax table.

Given a user's income, compute their tax burden by looking up the appropriate formula on the following table:

If taxable income is over--	But not over--	The tax is:
$0	$15,100	10% of the amount over $0
$15,100	$61,300	$1,510.00 plus 15% of the amount over 15,100
$61,300	$123,700	$8,440.00 plus 25% of the amount over 61,300
$123,700	$188,450	$24,040.00 plus 28% of the amount over 123,700
$188,450	$336,550	$42,170.00 plus 33% of the amount over 188,450
$336,550	no limit	$91,043.00 plus 35% of the amount over 336,550

First, we will introduce structure in the problem definition by dividing the process into discrete steps. We are entering the realm of pseudocode here, but there is still far too much natural language to be much of a help.

Determine tax bracket the user's income according to the ranges in this table:

Min	Max	Bracket
$0	$15,100	10%
$15,100	$61,300	15%
$61,300	$123,700	25%
$123,700	$188,450	28%
$188,450	$336,550	33%
$336,550	no limit	35%

Apply the appropriate formula:

Bracket	Formula
10%	10% of the amount over $0
15%	$1,510.00 plus 15% of the amount over $15,100
25%	$8,440.00 plus 25% of the amount over $61,300
28%	$24,040.00 plus 28% of the amount over $123,700
33%	$42,170.00 plus 33% of the amount over $188,450
35%	$91,043.00 plus 35% of the amount over $336,550

Return the results back to the caller.

Next we will be more precise with our equations and fill in more detail whenever possible. We are beginning to specify how things will be accomplished, not just what will be accomplished. Observe how our pseudocode is moving closer to our programming language with every step.

```
IF income is less than $15,100 then
    tax is 10% of the amount over $0
IF income between $15,100 and $61,300 then
    tax is $1,510 plus 15% of the amount over $15,100
IF income between $61,300 and $123,700 then
    tax is $8,440 plus 25% of the amount  over $61,300
IF income between $123,700 and $188,450 then
    tax is $24,040 plus 28% of the amount over $188,450
IF income is above $188,450 then
    tax is $91,043 plus 35% of the amount over $336,550
```

Finally, we reduce all operations to pseudocode keywords. Now we are ready to start writing code. The problem has been solved and now it is just a matter of translating the syntax-free pseudocode into the specific syntax of the high level language.

```
computeTax(income)

   IF ($0 ≤ income < $15,100)
      tax ← income * 0.10
   IF ($15,100 ≤ income < $61,300)
      tax ← $1,510 + 0.15 * (income - $15,100)
   IF ($61,300 ≤ income < $123,700)
      tax ← $8,440 + 0.25 * (income - $61,300)
   IF ($123,700 ≤ income < $188,450)
      tax ← $24,040 + 0.28 * (income - $123,700)
   IF ($188,450 ≤ income < $336,550)
      tax ← $42,170 + 0.33 * (income - $188,450)
   IF ($336,550 ≤ income)
      tax ← $91,043 + 0.35 * (income - $336,550)

   RETURN tax
END
```

Always remember that pseudocode is just a design tool. With the exception of a few assignments in this class, your purpose is to develop great software rather than write pseudocode. Therefore, you should use pseudocode only as far as it helps you to develop software. This means that sometimes you will design right down to the pseudocode keywords while other times you will be able to stop designing before that point because the solution presents itself.

Pseudocode keywords

There are seven classes of keywords: receive, send, math, remember, compare, repeat, and call functions.

Receive

A computer can receive information from a variety of input sources, such as keyboard, mouse, a network, a sensor, or wherever.

Keyword	Description	Example
READ	Receive information from a file	READ studentGrade
GET	Receive input from a user, typically from the keyboard	GET income

Send

A computer can send information to the console, display it graphically, write to a file, or operate on a device.

Keyword	Description	Example
PRINT	When sending data to a permanent output device like a printer	PRINT full student name
WRITE	When writing data to a file	WRITE record
PUT	When sending data to the screen	PUT instructions
PROMPT	Just like PUT except always preceding a GET instruction	PROMPT for user name

Arithmetic

Most processors have the built-in capability to perform the following operations:

Keyword	Description	Example
()	Parentheses are used to override the default order of operations	c = (f – 32) * 5/9
* x / ÷ mod div	Any mathematical convention can be used	numWeeks = days div 7
+ -	Addition / subtraction	SET count = count + 1
|| √ π	Common mathematical values or operations	PUT √10
≤ ≠	Common comparison operations	IF grade ≥ 60

Remember

A computer can assign a value to a variable or a memory location

Keyword	Description	Example
SET = ←	Assign a value to a variable ←	SET answer ← 42

Compare

A computer can compare two values and select one of two alternate actions

Keyword	Description	Example

IF
ELSE
	For two possible outcomes	IF income / 10 ≤ tithingPaid
		  PUT full tithe message
		ELSE
		  PUT scripture from Malachi

SWITCH CASE	For multiple possible outcomes. This will be discussed in more detail in Chapter 3.5	SWITCH option
		  CASE 1
		  PUT great choice!
		  CASE 2
		  PUT could be better
		  CASE 3
		  PUT please reconsider

Repeat

A computer can repeat a group of actions.

Keyword	Description	Example
WHILE	When repeating through the same code more than once	WHILE studentGrade < 60 takeClass()
FOR	When counting	FOR count = 1 to 10 by 2s PUT count

Functions

A computer can call a function and pass parameters between functions.

Usage	Conventions	Example

Declaring	Functions are named.
	Input parameters are enumerated.
	Statements are indented.
	RETURN values.
	END.
		computeTithing(income)
		  SET tithing = income / 10
		  RETURN tithing
		END

Calling	Call by name Specify parameters	PUT computeTithing(income)

Sue's Tips

On the surface, it may seem like pseudocode is no different than C++. Why learn another language when C++ already does the job? The short answer is that pseudocode is less detailed than C++ so you can concentrate on more high-level design decisions without getting bogged down in the minutia of detail that C++ demands. The long answer is a bit more complicated.

In its truest form, pseudocode is syntax free. This means that anything goes! It starts very free-form much like natural language (English). As you refine your thinking and work out the program details, it begins to take on the structure of a high-level programming language. When you get down to the pseudocode keywords listed above, you have worked out all the design decisions. While it is not always necessary to develop pseudocode to this level of detail, it is an important skill to develop. This is why pseudocode is an important CS 124 skill to learn.

Example 2.2 – Compute Pay

Demo

This example will demonstrate the relationship between pseudocode and C++. Specifically, it will show what kinds of details are necessary in pseudocode (variable names, equations, and program logic) and which are not (variable declarations, C++ syntax, and comments).

Problem

Write the pseudocode corresponding to the following C++ function:

```
/*********************************************************
 * COMPUTE PAY
 *   Based on the user's wage and hours worked, compute the pay
 *   taking into account time-and-a-half overtime
 *********************************************************/
float computePay(float hourlyWage, float hoursWorked)
{
   float pay;

   // regular rate
   if (hoursWorked < 40)
      pay = hoursWorked * hourlyWage;                  // regular rate

   // overtime rate
   else
      pay = (40.0 * hourlyWage) +                      // first 40 normal
         ((hoursWorked - 40.0) * (hourlyWage * 1.5));  // balance overtime

   return pay;
}
```

Solution

The pseudocode for `computePay()` is the following:

```
computePay(hourlyWage, hoursWorked)
   IF hoursWorked < 40
      SET pay = hoursWorked x hourlyWage
   ELSE
      SET pay = (40 x hourlyWage) + ((hoursWorked - 40) x (hourlyWage x 1.5)
   RETURN pay
END
```

Observe how the pseudocode completely represents the logic of the program without worrying about data-types, declaring variables, or the syntax of the language.

Challenge

As a challenge, look at the Project 1 definition. The pseudocode for most of the functions is provided. Compare your C++ code with the provided pseudocode. See if you can write the pseudocode for the remaining functions (`computeTithing()`, `getActualTax()`, etc.).

Problem 1

What is the value of b at the end of execution?

```
bool vegas(bool b)
{
   b = false;
   return true;
}

int main()
{
   bool b;
   vegas(b);
   return 0;
}
```

Answer:

_____

Please see page 65 for a hint.

Problem 2

What is the output when the user types 'a'?

```
void function(char &value)
{
   cin >> value;
   return;
}

int main()
{
   char input = 'b';
   function(input);
   cout << input << endl;
   return 0;
}
```

Answer:

_____

Please see page 65 for a hint.

Problem 3

Which of the following is not a basic computer operation?

- Receive information
- Connect to the network
- Perform math
- Compare two numbers

Please see page 145 for a hint.

Problem 4

What is wrong with the following pseudocode?

```
IF age < 18
PUT message about not being old enough to vote
```

Answer:

_____

Please see page 49 for a hint.

Problem 5

Which of the following is the correct pseudocode for sending a message to the user?

```
DISPLAY
```

```
cout << "The following are the instructions\n";
```

```
WRITE instructions on the screen
```

```
PUT instructions on the screen
```

Please see page 148 for a hint.

Problem 6

Which of the following is the correct pseudocode for computing time-and-a-half overtime?

```
pay = (hours - 40) * pay * 1.5 + 40 * pay
```

```
pay = ((float)hours - 40.0) * pay * 1.5 + 40 * (float)pay;
```

```
pay = hours - 40 * pay * 1.5 + 40 * pay
```

```
pay = hours over 40 times time-and-a-half plus regular pay for first 40 hours
```

Please see page 148 for a hint.

Problem 7

Which is the pseudocode command to differentiate between two options?

```
IF
```

```
COMPARE
```

```
=
```

```
same?
```

Please see page 149 for a hint.

Problem 8

Which pseudocode command displays data on the screen?

PUT

PROMPT

OUTPUT

SCREEN

Please see page 148 for a hint.

Problem 9

Write the pseudocode corresponding to the following C++:

```
float computeTithing(float income)
{
   float tithing;

   // Tithing is 10% of our income. Please
   // see D&C 119:4 for details or questions
   tithing = income * 0.10;

   return tithing;
}
```

Answer:

Please see page 49 for a hint.

Problem 10

Write the pseudocode for a function to determine if a given year is a leap year:

> According to the Gregorian calendar, which is the civil calendar in use today, years evenly divisible by 4 are leap years, with the exception of centurial years that are not evenly divisible by 400. Therefore, the years 1700, 1800, 1900 and 2100 are not leap years, but 1600, 2000, and 2400 are leap years.

Answer:

Please see page 150 for a hint.

Problem 11

Write the pseudocode for a function to compute how many days are in a given year. Hint: the answer is 365 or 366.:

Answer:

Please see page 150 for a hint.

Problem 12

Write the pseudocode for a function to display the multiples of 7 under 100.

Answer:

Please see page 148 for a hint.

Assignment 2.2

Please create pseudocode for the following functions. The pseudocode is to be turned in by hand in class for face-to-face students and submit it as a PDF for online students:

Part 1: Temperature Conversion

```
int main()
{
   // Get the temperature from the user
   float tempF = getTemp();

   // Do the conversion
   float tempC = (5.0 / 9.0) * (tempF - 32.0);

   // display the output
   // I don't want showpoint because I don't want to show a point!
   cout.setf(ios::fixed);
   cout.precision(0);
   cout << "Celsius: " << tempC << endl;

   return 0;
}
```

Part 2: Child tax credit

```
int main()
{
   // prompt for stats
   double income      = getIncome();
   int    numChildren = getNumChildren();

   // display message
   cout.setf(ios::fixed);
   cout.precision(2);
   cout << "Child Tax Credit: $ ";
   if (qualify(income))
      cout << 1000.0 * (float)numChildren << endl;
   else
      cout << 0.0 << endl;

   return 0;
}
```

Part 3: Cookie monster

```
void askForCookies()
{
   // start with no cookies  :-(
   int numCookies = 0;

   // loop until the little monster is satisfied
   while (numCookies < 4)
   {
      cout << "Daddy, how many cookies can I have? ";
      cin  >> numCookies;
   }

   // a gracious monster to be sure
   cout << "Thank you daddy!\n";
   return;
}
```

Please see page 150 for a hint.